

Appendix on Optimization Algorithm

Value ranges for various configurations of charging stations and solar panels:

- C-Parking:
 - + 11kW Charging Ports (7 values): 20, 25, 30, 35, 40, 45, 50
 - + 30kW Charging Ports (5 values): 2, 4, 6, 8, 10
- J-Parking:
 - + 11kW Charging Ports (6 values): 15, 18, 21, 24, 27, 30
 - + 30kW Charging Ports (4 values): 2, 4, 6, 8
- Solar Panels (8 values): 200, 300, 400, 500, 600, 700, 800, 900

Total Configuration

- C-parking wiht solar panels: **280** ($=7 \times 5 \times 8$);
- J-parking wiht solar panels: **6720** ($=7 \times 5 \times 6 \times 4 \times 8$);

Objective Function $f(x)$ in range $[0,3]$

Algorithm name	Source	Description	Pseudo Algorithm	Parameter and its value
Hill Climbing	(Pearl, 1984)	A local search that iteratively improves the solution.	Initialization of an initial solution s $iter = 0$ While $iter \leq \text{iter_max}$, do: Choice s' in the neighborhood of s maximize the fitness function If $f(s') > f(s) \Rightarrow s = s'$ Else $\Rightarrow$ end of the search process $iter = iter + 1$	(50) iter_max: Maximum number of iterations before stopping the exploration. This equals the number of "moves" in the parameter space. Each move tests the whole neighborhood of the current solution.

Simulated Annealing	(Kirkpatrick et al., 1983)	A probabilistic technique that avoids local optima.	Initialization of an initial solution s $temp = temp_init$ While $temp > temp_end$, do: $iter = 0$ While $iter < nb_iter_cst_temp$, do: Random choice of a solution $s2$ in the neighborhood of s $df = f(s2) - f(s)$ If $df > 0 \Rightarrow s = s2$ Else, $rand = \text{random number between } 0 \text{ and } 1$ If $rand < \exp(df/temp) \Rightarrow s = s2$ $iter = iter + 1$ $temp = temp * temp_decrease$	(0.5) temp_init: Initial temperature. Represents the probability of accepting a worse solution during the initial search phase. Higher values increase the chance of escaping local optima (e.g., 0.5 ~ 17% range $f(x) = 3$). (0.02) temp_end: Final temperature. A lower final temperature ensures precision in final search stage. (0.95) temp_decrease: Temperature decrease coefficient. Governs the rate of cooling (closer to 1 means slower cooling, which allows for better exploration of the solution space). (5) nb_iter_cst_temp: Number of iterations per temperature level. A smaller number helps save computation time.
Tabu Search	(Glover, 1989, 1990)	Uses memory structures to avoid revisiting solutions.	Initialization of an initial solution s $tabuList = \{\}$; $iter = 0$ While $iter \leq iter_max$, do: Choice of the solution $s2$ in the neighborhood of s that: $s2$ is not in $tabuList$ & the fitness function is maximal for $s2$ $s = s2$ If size of $tabuList = tabu_list_size$ removing of the oldest solution in $tabuList$ $tabuList = tabuList + s$ $iter = iter + 1$	(280) iter_max: Maximum number of iterations before stopping the exploration. (3) tabu_list_size: Size of the tabu list.

Reactive Tabu Search	(Battiti and Tecchiolli, 1994)	Similar to Tabu Search but adjusts memory size for improved efficiency.	Initialization of an initial solution s $\text{tabuList} = \{\}$; $\text{tabu_list_size} = \text{tabu_list_size_ini}$ $\text{iter} = 0$; $\text{nb_tests_without_col} = 0$ While $\text{iter} \leq \text{iter_max}$, do: Choice of the solution s_2 in the neighborhood of s such that: s_2 is not in tabuList & the fitness function is maximal for s_2 $s = s_2$ If $\text{size of tabuList} \geq \text{tabu_list_size}$ Removing the oldest solution in tabuList $\text{tabuList} = \text{tabuList} + s$ If $\text{nb_tests_without_col} > \text{nb_tests_without_col_max}$ If $\text{tabu_list_size} > \text{tabu_list_size_min} \Rightarrow$ Reduce tabu_list_size $\text{nb_tests_without_col} = 0$ Else $\Rightarrow \text{nb_tests_without_col} = \text{nb_tests_without_col} + 1$ If significant improvement in fitness is detected If $\text{tabu_list_size} < \text{tabu_list_size_max} \Rightarrow$ Increase tabu_list_size If a cycle of size within $[\text{cycle_size_min}, \text{cycle_size_max}]$ is detected Restart with a different initial solution $\text{iter} = \text{iter} + 1$	(140) iter_max: Maximum number of iterations. (10) tabu_list_size_ini: Initial size of the tabu list. (5) tabu_list_size_min: Minimal size of tabu list. (20) tabu_list_size_max: Maximal size of tabu list. (20) nb_tests_without_col_max: Maximum number of movements without collision (improving move) before the tabu list size is shortened. (3) cycle_size_min: Minimal size of considered cycles (for aspiration criterion). (10) cycle_size_max: Maximal size of considered cycles (for aspiration criterion).
---------------------------------	--------------------------------------	--	--	---

Genetic Algorithm	(Holland, 1975; Goldberg and Holland, 1988)	A population-based algorithm inspired by evolution.	Initialization of a population P of size pop_dim using nb_prelim_gen random populations Evaluation of each individual in P gen = 0 While gen <= max_gen, do: Selection of parent individuals based on fitness Crossover with probability crossover_prob: Combine parts of parents to generate offspring Mutation with probability mutation_prob: Randomly modify some genes in the offspring Evaluation of offspring Replacement: Replace the least fit individuals in P with the new offspring gen = gen + 1 Return the best solution in P	(20) pop_dim: Size of the population (number of individual solutions). (0.8) crossover_prob: Crossover probability between two individual solutions. The likelihood of combining two parents to create a new offspring. (0.1) mutation_prob: Mutation probability for an individual solution. This is the likelihood of changing a small part of an individual (3) nb_prelim_gen: Number of random populations used to build the initial population. (40) max_gen: Maximum number of generations.
Particle Swarm Optimization	(Kennedy and Eberhart, 1995)	A swarm intelligence method where particles adjust positions based on experience.	Initialization a swarm of num_particles particles {p1, p2, ..., pN}, with: Initial positions xi and velocities vi Personal best positions pi = xi Global best position g = argmax(f(pi)) iter = 0 While iter <= iter_max, do: For each particle i in the swarm: Update velocity: vi = weight_inertia * vi + weight_cognitive * rand1 * (pi - xi) + weight_social * rand2 * (g - xi) Update position: xi = xi + vi If f(xi) > f(pi) => pi = xi If f(pi) > f(g) => g = pi iter = iter + 1 Return g as the best solution	(10) iter_max: Maximum number of iterations. (5) num_particles: Number of particles in swarm. (1.4) weight_cognitive: Weight for the cognitive component. A large value prioritizes the particle's individual best experience, potentially leading to stagnation in a local optimum. (0.7) weight_inertia: Weight for the inertia component. This determines the extent to which a particle retains its previous velocity. (1.6) weight_social: Weight for the social component. This defines the degree to which the particle is attracted to the group's best position.